

Click-to-Source

Chosen Approach: NPM Developer Tool + Built-In MCP/AI Mode

Final Architecture & Development Plan | Punith P | August 2026

This document describes the finalized approach for Click-to-Source: a single npm developer tool, installed directly into a React Three Fiber project, that resolves a clicked rendered feature back to its exact source code and generator arguments — with an optional built-in mode that exposes the same resolution engine to AI coding agents via the Model Context Protocol (MCP). It covers why this shape was chosen, how it works technically, and what each stage of building it involves.

How This Differs From Needle Inspector

Needle Inspector is the closest existing tool to this idea, and it's a strong, mature product — worth being precise about where it overlaps and where it doesn't, rather than vaguely claiming novelty.

Needle answers **"what is this, and how does it currently look or perform?"** — scene hierarchy, live property editing, shader/node graphs, memory and performance profiling, AI chat that can see and tweak the running scene. It is excellent at showing the *result* of code: this mesh has this material, this many triangles, this shader graph.

This tool answers a different question: **"why does this exist, and what exact code and values produced it?"** — not the current state of an object, but the causal chain: which function created this specific tree, with which exact arguments, out of the hundreds of other trees that same function also created.

	Needle Inspector	This Tool (Click-to-Source)
Core question	What is this object, right now?	Why does this object exist, and what exact code/arguments produced it?
Mechanism	Runtime scene-graph hooking + standard browser source maps.	Build-time AST instrumentation (Vite/Babel) tagging generator calls with provenance.
Editing model	Live runtime property tweaks (position, material, color) — exportable as a changeset for later manual reconciliation with code.	Direct edits to the generator function's source, so the fix is permanent and regenerates correctly for every future instance.
"Fix one lake" vs.	Fixes the one lake instance you're looking at, in that session.	Fixes the function that makes every lake, everywhere it's called.
Per-instance args	Not exposed — no documented mapping from an object back to the generator call or arguments that created it.	The core feature — exact runtime arguments (seed, scale, height) captured per instance.
Scope	Any three.js/R3F site, including production sites you didn't build.	Your own project, in your own dev environment — by design, not a limitation (see below).

The gap is structural, not a missing checkbox. Needle's own documentation and MCP tool set are silent on generator-function or argument-level tracing anywhere — not because it's an oversight, but because their architecture (runtime hooking + source maps) has no path to it. Reaching that capability requires build-time instrumentation, a fundamentally different mechanism Needle doesn't currently use for the inspector. That's why this is treated as a genuine gap rather than a feature request Needle might ship next.

On scope, by design: Needle's research explicitly recommends a browser-extension model, since its goal is working on any site including ones you didn't build. That reasoning doesn't apply here — this tool's entire purpose is locating your own code in your own dev environment, so an npm package that ships with the project remains the right call even though it means the tool won't work on arbitrary third-party sites. That's an intentional trade, not an oversight.

Why This Shape, Specifically

This is a developer tool for working on your own project in your own development environment — not a general-purpose inspector meant to run on arbitrary third-party sites. That distinction is what settles the architecture:

- **The intended use is locating your own code** — the code behind a mountain ridge, a shader, a weather particle system you are actively building. There is no scenario where inspecting a stranger's live site helps with that, since you don't have their source to edit even if you could see their scene graph. "Works on any website" is a feature that matters for a different kind of product, not this one.
- **A single npm package keeps the tool coherent.** One codebase, one install step, one mental model — instead of maintaining a browser extension, an IDE plugin, and a package as three separate surfaces with three separate release cycles.
- **AI-agent access is a mode of the same tool, not a separate product.** The same resolution engine that powers the click-to-inspect overlay can be exposed to an AI coding agent through MCP, using the identical raycast-and-resolve logic. No second codebase to maintain.

The Core Engine (Shared by Both Modes)

Everything in this tool sits on top of one shared pipeline. Both the visual overlay and the MCP/AI mode are just two different front doors into the same engine. Following the terminology used across similar tooling research, the captured record is referred to as **provenance** — where an object came from, not just what it currently is:

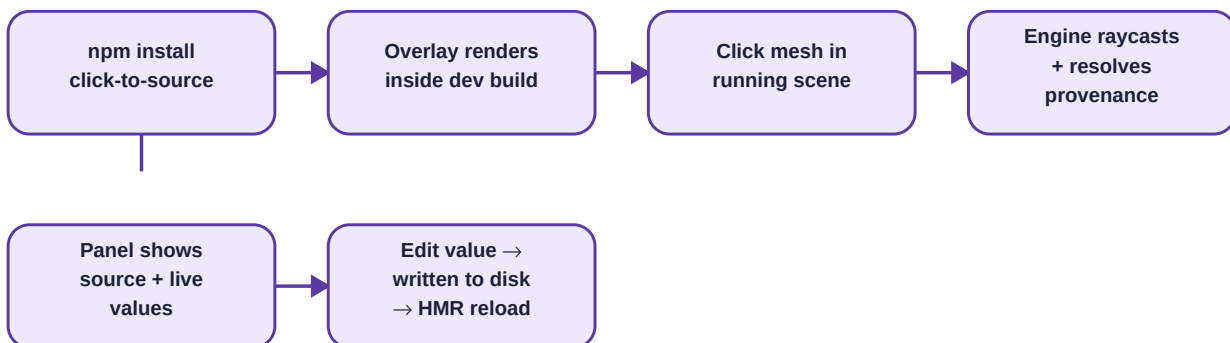
Runtime Object → **Provenance** → **Source File** → **Exact Arguments**

- **Provenance capture:** generator functions (e.g. `createTree()`, `generateTerrainPatch()`) are tagged so that whatever object they return carries a record of where it was made and what arguments produced it. v0.1 uses manual tagging (`mesh.userData.sourceRef`); a later build-time Vite/Babel transform automates this so no manual annotation is needed.
- **Raycasting:** a click (or an AI-issued coordinate/object-name query) is translated into a `THREE.Raycaster` call against the live scene, identifying the exact mesh involved.
- **Resolution:** the identified mesh's provenance record is read back — file, function, line, and the exact runtime argument values used for that specific instance, not just the line that ran.
- **Source access:** since this all runs inside the developer's own local dev server, reading and writing the actual source file is a same-machine file operation — no cross-origin or filesystem-permission issues to work around.

Because both modes are built on the same four-step pipeline, building the resolution engine well once is what makes both the overlay and the AI integration possible — neither mode duplicates this logic.

Mode 1 — Visual Overlay (Human-Facing)

The default mode. A developer imports the package into their React Three Fiber app; it renders a small overlay panel — a **Generation Trace** panel — active only in development builds.

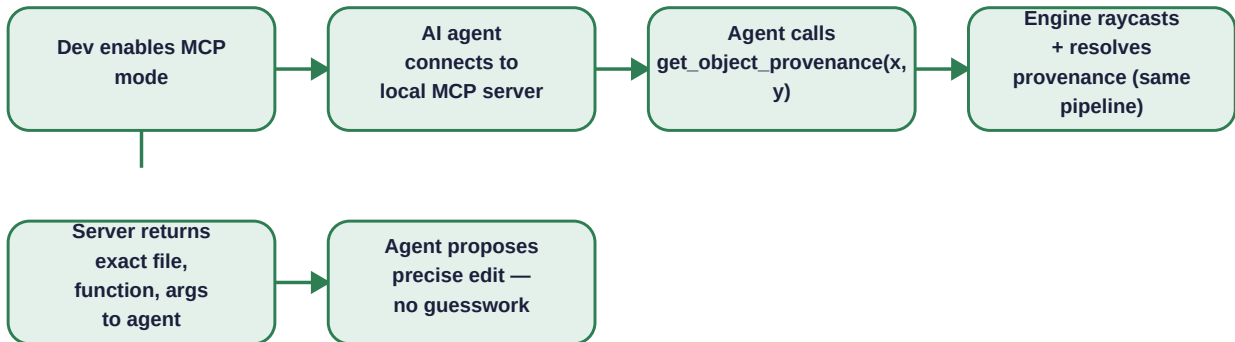


What the developer sees:

- A highlighted outline around the clicked mesh (via a Three.js outline pass or bounding-box helper).
- A Generation Trace panel showing the resolved file, function name, and the exact argument values for that instance.
- Editable fields for those values — changing one writes the new value back to the source file and lets the existing Vite dev-server HMR pipeline reload the scene, with no custom hot-reload logic required.

Mode 2 — MCP / AI Agent Access (Optional, Built-In)

Enabled with a flag or config option (e.g. `click-to-source --mcp`) — starts a small local MCP server alongside the normal dev server, exposing the same engine as callable tools for AI coding agents (Claude Code, Cursor, Claude Desktop, etc.).



Indicative tool surface exposed to the agent:

```
get_object_provenance(x, y) → { file, function, line, args }
get_source(file, line) → source snippet
edit_parameter(file, line, argName, newValue) → writes edit, triggers HMR
search_by_seed(seed) → all instances generated with a given seed
search_by_generator(functionName) → all instances produced by a given generator function
```

The last two — searching by seed or by generator across the whole scene, not just resolving one clicked object — go beyond what a click-driven UI alone can offer, and are meaningfully more useful to an AI agent reasoning about a whole class of objects at once (e.g. "fix every tree placed by this generator").

Why this matters for the original token-saving goal:

- Without this tool, asking an AI to fix "that lake that looks too deep" requires the AI to search the codebase, guess at the right variable, and often iterate — each round trip costs tokens and time.
- With MCP mode, the exact file/function/argument is resolved deterministically before the AI is ever asked to reason about it — the AI receives precise context instead of having to search for it.
- This is the same engine, the same provenance data, just handed to an agent instead of displayed in a panel — confirming this is one tool with two interfaces, not two tools.

Known Risks

Specific technical risks worth naming plainly rather than discovering mid-build:

- **Ambiguous provenance.** Not every object has one clean generator. An object created inside a shared utility function called from many places, or mutated after creation by multiple systems (physics, layout, animation), may not resolve to a single obvious "this is what made it." Stage 1 exists specifically to find out how common this is before committing to the full build.
- **Instrumentation overhead.** AST-level wrapping of generator functions can increase bundle size, interfere with tree-shaking, and complicate HMR if implemented carelessly. This is a reason to keep the instrumentation scoped narrowly (only functions explicitly opted in) rather than wrapping broadly.
- **R3F reconciler brittleness.** Mapping a raw `Object3D` back to the specific React Three Fiber component and props that created it means touching R3F's internal fiber tree, which isn't a stable public API and could break across R3F version upgrades. The v0.1 scope (manual tagging, meshes only) sidesteps this risk; it only resurfaces if the tool later tries to resolve all the way back to JSX/component level rather than stopping at the generator-function level.

Development Stages

Brief description of each stage. Both modes share the same core engine, so MCP support is added once (Stage 6) after the engine and overlay are proven — not built in parallel from day one.

Stage	What Happens	Est. Time
0 — Scope freeze	Lock the one-sentence definition: an npm dev tool that resolves a clicked scene feature to its exact source and arguments, with an optional MCP mode. Anything outside that waits.	< 1 day
1 — Prove the mechanism	Throwaway prototype on the existing Procedural World Generator: manually tag one generator function, raycast a click against it, confirm metadata resolves correctly. Validates the core idea before any UI is built.	2–3 days
2 — Metadata convention	Define the manual tagging convention (<code>userData.sourceRef</code>) used for v0.1. Design, but don't yet build, the future Vite/Babel auto-instrumentation transform.	~1 week
3 — Core engine + overlay (MVP)	Build the shared engine (raycast → resolve → metadata) and the visual overlay UI on top of it: click, see source + values, edit, live-update. No AI yet.	2–3 weeks
4 — Dogfooding	Use the tool on the real Procedural World Generator while fixing its known regressions (normals ordering, weather freeze, lake bed height clamp). Real-world proof, not a synthetic demo.	1–2 weeks
5 — Package polish	Clean install experience, sensible defaults, README framed around the actual problem (AI gets you 90% there, this finds the last 10%), short demo GIF/video.	~1 week
6 — MCP / AI mode	Add the optional MCP server mode on top of the now-proven engine: expose <code>get_object_provenance</code> , <code>get_source</code> , <code>edit_parameter</code> , <code>search_by_seed</code> , and <code>search_by_generator</code> as agent-callable tools. Reuses Stage 3's engine entirely.	~1 week
7 — Ship	Publish to npm and GitHub (MIT license), short demo video showing both modes, narrow launch to a relevant community (<code>r/threejs</code> , <code>r/webdev</code> , relevant Discords).	Ongoing

Total realistic timeline: roughly 7–10 weeks of consistent part-time work, assuming Stage 1 doesn't reveal a fundamental blocker in the mapping mechanism. This is slightly longer than an overlay-only plan because Stage 6 (MCP mode) is now included as a committed feature rather than a deferred maybe.

Success Criteria

Overlay mode: within 30 seconds of clicking an object, a developer who did not write the scene can find and edit the parameter responsible for that object's appearance.

MCP mode: given a vague visual description of a problem ("the lake looks too deep"), an AI agent using the tool resolves the exact file/function/argument in a single tool call, with no back-and-forth searching — a measurable reduction in iteration count and tokens compared to the same request without the tool.

Prepared by Punith P — Computer Science & Engineering, KS Institute of Technology, Bengaluru. Companion document to the Click-to-Source Project Proposal and Technical Approaches comparison.